

Battery Builder API

```
% =====  
% PHASE IV & V: INTEGRATED BATTERY BUILDER (CORRECTED)  
% Task: Build a 75kWh High-Voltage Pack (96S 46P) for EV Verification  
% =====  
clearvars;  
clc;  
import Simscape.Battery.Builder.*  
  
% --- STEP 1: LOAD PARAMETERS (ROBUST DEFAULTS) -----  
disp('[STEP 1] Loading/Defining Parameters...');  
  
[STEP 1] Loading/Defining Parameters...  
  
data_path = 'C:\Users\Lenovo\Desktop\Battery_Pack_Design_Project_142\Data';  
data_file = fullfile(data_path, 'estimated_params.mat');  
  
if exist(data_file, 'file')  
    load(data_file);  
    % OVERRIDE CAPACITY for EV Scale (if the file has old 2Ah data)  
    Cap_val = 4.8;  
else  
    disp('[WARN] Data file not found. Using Standard 21700 NMC Defaults.');    R0_val = 0.025; R1_val = 0.001; C1_val = 2000; % Lower resistance for EV cell  
    R2_val = 0.03; C2_val = 5000;  
    R3_val = 0.005; C3_val = 1000;  
    Cap_val = 4.8; % 4.8 Ah (21700 Form Factor)  
end  
  
% --- STEP 2: DEFINE GEOMETRY & PHYSICS -----  
% 1. Geometry (21700 Standard)  
batt_geom = batteryCylindricalGeometry;  
batt_geom.Radius = Simscape.Value(10.5, 'mm'); % 21mm Diameter  
batt_geom.Height = Simscape.Value(70, 'mm'); % 70mm Height  
  
cell_obj = batteryCell(batt_geom);  
cell_obj.Mass = Simscape.Value(0.068, 'kg'); % Approx 68g  
  
% 2. Physics (3RC Equivalent Circuit)  
params = struct();  
params.thermal_port = 'model';  
params.R0_Main = Simscape.Value(R0_val, 'Ohm');  
params.R1_Main = Simscape.Value(R1_val, 'Ohm');  
params.C1_Main = Simscape.Value(C1_val, 'F');  
params.R2_Main = Simscape.Value(R2_val, 'Ohm');  
params.C2_Main = Simscape.Value(C2_val, 'F');  
params.R3_Main = Simscape.Value(R3_val, 'Ohm');  
params.C3_Main = Simscape.Value(C3_val, 'F');  
params.capacity = Simscape.Value(Cap_val, 'A*hr'); % 4.8 Ah  
params.soc_start = Simscape.Value(0.95, '1'); % Start at 95% Charge
```

```
cell_obj.CellModelOptions.BlockParameters = params;
```

```
% --- STEP 3: DEFINE ARCHITECTURE (EV SCALE) -----  
disp(['STEP 3] Defining High-Voltage Architecture...');
```

```
[STEP 3] Defining High-Voltage Architecture...
```

```
% 1. Parallel Assembly (38P) -> Capacity
```

```
% 4.8Ah * 38 = 182,4 Ah Pack Capacity
```

```
p_assembly = batteryParallelAssembly(cell_obj, 38, ...  
    'Topology', 'Hexagonal', 'ModelResolution', 'Lumped', ...  
    'AmbientThermalPath', 'CellBasedThermalResistance');
```

```
% 2. Module (90S) -> Voltage
```

```
% 3.7V * 90 = 333 V Nominal Voltage
```

```
module_obj = batteryModule(p_assembly, 90, ...  
    'SeriesGrouping', 90, ...  
    'ModelResolution', 'Grouped', ...  
    'AmbientThermalPath', 'CellBasedThermalResistance');
```

```
% 3. Module Assembly & Pack
```

```
ma_obj = batteryModuleAssembly(module_obj, ...  
    'AmbientThermalPath', 'CellBasedThermalResistance');
```

```
pack_obj = batteryPack(ma_obj, ...  
    'AmbientThermalPath', 'CellBasedThermalResistance');
```

```
% --- STEP 4: FORCE LIBRARY REBUILD (CRITICAL) -----
```

```
libName = 'MyAutomatedPackLib';  
libFile = [libName, '_lib.slx'];
```

```
if exist(libFile, 'file') == 4  
    disp('=====');  
    disp(['[INFO] Deleting Old Library: ', libFile]);
```

```
    % Unlock and Close
```

```
    try bdclose([libName, '_lib']); catch; end
```

```
    % Delete
```

```
    delete(libFile);
```

```
    disp('        Old library deleted to force update.');
```

```
    disp('=====');
```

```
end
```

```
% --- STEP 5: BUILD -----
```

```
disp(['[INFO] Building New High-Voltage (96S-46P) Library...']);
```

```
buildBattery(pack_obj, 'LibraryName', libName);
```

```
addpath(pwd);
```

```
disp('=====');
```

```

disp('[SUCCESS] Library Rebuilt.');
```

- disp('ACTION REQUIRED:');
- disp(' 1. Open Drive_Cycle_Val.slx');
- disp(' 2. DELETE the old Pack1 block.');
- disp(' 3. Drag the NEW Pack block from MyAutomatedPackLib.');
- disp(' 4. Re-run Simulation.');
- disp('=====');

Verification & Validation

```

% =====
% PHASE V: ARCHITECTURE VALIDATION & DIMENSIONAL ANALYSIS
% Task: Verify Library Generation, Voltage, Capacity, and Mass
% =====
clc;
disp('-----');
disp('          PHASE V: VERIFICATION & VALIDATION STARTING...');
disp('-----');

% --- CHECK 1: DEPENDENCY CHECK -----
% We need the 'pack_obj' from the Phase IV script to perform analysis.
if ~exist('pack_obj', 'var')
    error('[ERROR] "pack_obj" not found in workspace! Please run the "Phase IV: Battery Builder" script first.');
```

```

else
    disp('[STEP 1] Workspace check: "pack_obj" found. Proceeding...');
end

% --- CHECK 2: LIBRARY GENERATION CHECK -----
libName = 'MyAutomatedPackLib';
libFile = [libName, '_lib.slx'];

disp(['[STEP 2] Inspecting Library: ', libFile]);

if exist(libFile, 'file') == 4
    try
        % Open the system (hidden) to inspect contents
        load_system(libFile);

        % Find all blocks at the top level of the library
        blocks = find_system(libName, 'SearchDepth', 1, 'Type', 'Block');

        % Remove the library handle itself from the list
        blocks = setdiff(blocks, libName);

        % Check if we have exactly 1 block (The Pack)
        if isscalar(blocks)
```

```

        disp(['    [PASS] Library contains exactly 1 block: ', char(blocks)]);
    else
        disp(['    [WARN] Library contains ', num2str(numel(blocks)), ' blocks.
Expected 1.']);
        disp(blocks);
    end

    % Close library (optional, keeps workspace clean)
    % close_system(libName);

    catch ME
        disp(['    [FAIL] Could not inspect library: ', ME.message]);
    end
else
    error('[FAIL] Library file does not exist. Build failed?');
end

% --- CHECK 3: DIMENSIONAL ANALYSIS (VOLTAGE) -----
disp(' ');
disp('[STEP 3] Verifying Voltage Architecture (Target: ~355.2 V)');

% Navigate hierarchy to find series counts
% Hierarchy: Pack -> ModuleAssembly -> Module -> ParallelAssembly
try
    % 1. Cells in Series per Parallel Assembly (Usually 1 for 18650s)
    n_series_cell = pack_obj.ModuleAssembly.Module.ParallelAssembly.NumSeriesCells;

    % 2. Series Assemblies per Module (The "96" we set)
    n_series_module = pack_obj.ModuleAssembly.Module.NumSeriesAssemblies;

    % 3. Modules in Series (Usually 1 in this Giant Module architecture)
    n_series_assemblies = pack_obj.ModuleAssembly.NumModuleAssemblies;

    % Calculate Total Series Count (Ns)
    Ns_total = n_series_cell * n_series_module * n_series_assemblies;

    % Nominal Voltage (3.7V per cell)
    v_nominal_pack = Ns_total * 3.7;

    fprintf('    > Series Cells (Depth):      %d\n', n_series_cell);
    fprintf('    > Series Assemblies (Rows):    %d\n', n_series_module);
    fprintf('    > Modules in Series:           %d\n', n_series_assemblies);
    fprintf('    > Total Series (Ns):           %d\n', Ns_total);
    fprintf('    > Calculated Voltage:          %.2f V\n', v_nominal_pack);

    if abs(v_nominal_pack - 355.2) < 5.0
        disp('    [RESULT] VOLTAGE CHECK: PASS');
    else
        disp('    [RESULT] VOLTAGE CHECK: FAIL (Mismatch > 5V)');
    end
end

```

```

catch
    disp('    [FAIL] Structure mismatch. Could not parse pack_obj hierarchy.');
```

end

```

% --- CHECK 4: DIMENSIONAL ANALYSIS (CAPACITY) -----
disp(' ');
disp('[STEP 4] Verifying Capacity (Target: 60 Ah)');
```

try

```

    % 1. Parallel Cells (The "30" we set)
    n_parallel = pack_obj.ModuleAssembly.Module.ParallelAssembly.NumParallelCells;

    % 2. Cell Capacity (The "4.8 Ah" we loaded)
    % Note: We extract the value safely using Simscape 'value' function if needed
    if
isprop(pack_obj.ModuleAssembly.Module.ParallelAssembly.Cell.CellModelOptions.BlockParameters, 'capacity')
        % Struct method
        cell_cap_val =
value(pack_obj.ModuleAssembly.Module.ParallelAssembly.Cell.CellModelOptions.BlockParameters.capacity, 'A*hr');
```

else

```

        % Fallback / Assumption if property is hidden
        cell_cap_val = 4.8;
    end

    % Calculate Total Capacity
    cap_total = n_parallel * cell_cap_val;

    fprintf('    > Parallel Cells (Np):        %d\n', n_parallel);
    fprintf('    > Cell Capacity:                    %.2f Ah\n', cell_cap_val);
    fprintf('    > Calculated Pack Capacity:         %.2f Ah\n', cap_total);

    if abs(cap_total - 60.0) < 0.1
        disp('    [RESULT] CAPACITY CHECK: PASS');
```

else

```

        disp('    [RESULT] CAPACITY CHECK: FAIL');
```

end

catch

```

    disp('    [FAIL] Could not calculate capacity.');
```

end

```

% --- CHECK 5: DIMENSIONAL ANALYSIS (MASS) -----
disp(' ');
disp('[STEP 5] Verifying Mass Calculation');
```

try

```

    % 1. Total Cell Count
```

```

N_total_cells = Ns_total * n_parallel;

% 2. Single Cell Mass (0.045 kg)
cell_mass = value(pack_obj.ModuleAssembly.Module.ParallelAssembly.Cell.Mass,
'kg');

% 3. Theoretical Mass (Cells Only)
mass_theoretical = N_total_cells * cell_mass;

% 4. Actual Simscape Object Mass (Includes housings/overhead if configured)
mass_simscape = value(pack_obj.Mass, 'kg');

fprintf('    > Total Cell Count:           %d\n', N_total_cells);
fprintf('    > Single Cell Mass:           %.4f kg\n', cell_mass);
fprintf('    > Theoretical Mass (Cells):    %.2f kg\n', mass_theoretical);
fprintf('    > Simscape Object Mass:        %.2f kg\n', mass_simscape);

if abs(mass_simscape - mass_theoretical) < 1.0
    disp('    [RESULT] MASS CHECK: PASS (Matches cell sum exactly)');
else
    disp('    [RESULT] MASS CHECK: NOTE (Simscape includes packaging overhead)');
end

catch
    disp('    [FAIL] Could not calculate mass.');
```

```

end

disp(' ');
disp('-----');
disp('          VERIFICATION COMPLETE: READY FOR SIMULATION          ');
disp('-----');
```